

Week 2 Assignment — Days 6-10

Practical Cyber Security (v2)

September 22, 2026

CONTENTS

Week 2 Assignment — Days 6–10	
How the week fits together	
The pattern for every day	
Day 6 — Files, the filesystem tree, and the shell · due start of Day 7	
Day 7 — Permissions, users, and sudo · due start of Day 8	
Day 8 — Windows: same question, different machine · due start of Day 9	
Day 9 — Scripting for security · due start of Day 10	
Day 10 — Security thinking · in-class exercise, no separate homework	
Weekend assignment — Week 2	
Part A — Integrate (harden your machine)	
Part B — R&D stretch (pick one)	
Part C — Hands-on evidence	
Part D — Reflection	
Part E — OverTheWire Bandit & command mastery	
How the weekend assignment is marked	
Submission checklist	

Week 2 Assignment — Days 6–10

Practical Cyber Security (v2) · Week 2: Understanding the operating system

This handout is everything you have to **do and hand in** for Week 2: one homework task after each class (Days 6–9), and the Day 10 weekend assignment. Class notes, recaps and key terms are in your Student Pack.

Prerequisite: you need a **Linux shell** every day this week (set up as Day 5 homework — WSL / a provided VM / the class Linux box; not Git Bash).

How the week fits together

Each day answers one question. The weekend assignment puts the whole week's answers to work on a real machine.

Day	The question	The path
6	How do you move around and read a Linux system?	filesystem tree → paths → shell → read / find / grep → pipes
7	Who can do what on this machine?	<code>rwX</code> → owner/group/other → <code>chmod</code> / <code>chown</code> → root/ <code>sudo</code> → privilege escalation
8	Same question, different machine — how does Windows control access?	SIDs/tokens → admin vs standard + UAC → NTFS ACLs → registry → PowerShell
9	How do you make the machine do the repetitive work?	5 building blocks → Bash → Python → <code>scanner.py</code>
10	How do you think like an attacker, systematically?	asset/vuln/threat/risk → attack surface → threat modelling (STRIDE)
Weekend	Can you harden a real machine and prove it — and go further on your own?	threat-model → harden → script → evidence + Bandit + command mastery

The pattern for every day

Every homework follows the same six steps, so you always know what you are doing and why:

Learn (class) → **Do** (build or run it) → **Investigate** (look at the real thing) → **Secure** (answer the day's security question) → **Evidence** (prove it on your machine) → **Commit** (push to your repo).

Every day also ends with one **Real job** question: where would you meet this in a real security job? Two or three sentences is plenty.

Your course repo. Same repo from Week 1 — after every class you add today's attack, defence, and artifact. Week 2 adds `day06/` through `day10/`.

When things are due

Task	Due
Day 6 homework	Start of Day 7
Day 7 homework	Start of Day 8
Day 8 homework	Start of Day 9
Day 9 homework	Start of Day 10
Weekend assignment, Parts A–D	Before Monday, start of Day 11 (one PDF)
Part E — Bandit + command explainer	Before Monday, start of Day 11 — separate submission , but start it Day 6, not the weekend

Day 6 — Files, the filesystem tree, and the shell · due start of Day 7

Security question: With nothing but a shell and `ls` / `cat` / `grep` / `find`, how does an intruder map a machine they just landed on — and what's the defence?

Do — start your Linux cheat-sheet. Copy `day06/assets/linux-cheatsheet-template.md` into your repo as `day06/linux-cheatsheet.md` and fill in **15 commands**, one line each, of what each does. Commit it — you'll keep adding to it all week.

Investigate — scavenger hunt. Write the command **and** the answer for each:

1. How many accounts have `/bin/bash` as their shell?
2. What is the largest file under `/var/log` ?
3. Which line of `/etc/ssh/sshd_config` mentions `Port` ? (no `sshd_config` ? use any file in `/etc` and say which.)

Secure — answer the security question in 2–3 sentences. Use class ideas: everything an attacker does right after getting a shell starts with exactly these commands; the defence is least privilege on files (tomorrow), no secrets in plain files or shell history, logs shipped off-box.

Evidence — the three scavenger-hunt answers with their commands, and your cheat-sheet commit.

Commit — into `day06/`, and update your “today I learned” list.

Real job — Where would a SOC analyst or a penetration tester use exactly these four commands (`ls`, `cat`, `grep`, `find`) on a real engagement?

Day 7 — Permissions, users, and `sudo` · due start of Day 8

Security question: How does a low-privilege Linux user become root, and what closes each path?

Do — write the artifact. “3 ways a low-privilege Linux user could become root, and the fix for each” — in your own words (SUID abuse, a writable script root/`cron` trusts, a loose `sudo` rule, or a PATH hijack). Commit it.

Investigate

1. Create `secret.txt`, set it so **only you** can read it. Show the `ls -l` line and the `chmod` you used.
2. Run `find / -perm -4000 -type f 2>/dev/null`. Pick **one** SUID program and, in one sentence, say why it legitimately needs to be SUID.

Secure — in 2–3 sentences: why is `chmod 777` never the fix, and what should you do instead when something “needs” wide-open permissions?

Evidence — the `ls -l` / `chmod` pair for `secret.txt`, and the SUID list with your chosen program named.

Commit — into `day07/`, and add `chmod`, `chown`, `sudo`, `id`, `ps`, `kill`, `systemctl`, `crontab` to your cheat-sheet.

Real job — Why would a penetration tester run `find / -perm -4000 -type f 2>/dev/null` within minutes of getting a shell on a target?

Day 8 — Windows: same question, different machine · due start of Day 9

Security question: Are you running as an administrator right now, and how would a defender tell the difference between your session and one that’s been quietly escalated?

Do — build the Linux ↔ Windows command table. At least **5 rows**, a real command on both sides (e.g. `id` ↔ `whoami /groups`, `ls -l` ↔ `icacls`, `ps` ↔ `Get-Process`). Commit it.

Investigate

1. Run `whoami /priv` and `whoami /groups`. In two sentences: are you an administrator, and how can you tell?
2. Run `Get-ItemProperty` on **both** Run keys (`HKCU:\...\Run` and `HKLM:\...\Run`). List what auto-starts on your machine; flag anything you can’t identify — **don’t delete anything**.

Secure — in 2–3 sentences: why is a Run key a **persistence** mechanism, and what's the defence (least privilege day-to-day, monitoring autoruns, EDR)?

Evidence — the `Get-ItemProperty` output for both Run keys, and the command table.

Commit — into `day08/`. Copy `day08/assets/windows-cheatsheet-template.md` into your repo as `day08/windows-cheatsheet.md` and add `whoami`, `Get-LocalUser`, `Get-Process`, `Get-Service`, `icacils`, `Get-ItemProperty` to it.

No Windows machine? Use the class lab VM or pair with a neighbour, and say which in your evidence.

Real job — Why do incident responders check Run keys, Services, and Scheduled Tasks first when they suspect a Windows machine has been compromised?

Day 9 — Scripting for security · due start of Day 10

Security question: Attackers and defenders both automate. Why is it the *same* five building blocks either way?

Do — commit `scanner.py` and `sweep.sh`, each with a `# usage:` comment at the top.

Investigate

1. Extend `scanner.py` to read target host(s) from `hosts.txt` instead of hardcoding `127.0.0.1`.
2. Read `day09/assets/ai_snippet.py`. In 2–3 sentences: what does it do, and what's wrong with it? (There's more than one problem.)

Secure — in 2–3 sentences: “you own what you run” — what's the actual risk of running LLM-generated code you haven't read line by line?

Evidence — `scanner.py`'s output run against `127.0.0.1` (and the class lab host if you have one), and your `ai_snippet.py` critique.

Commit — into `day09/`, and add `for`, `if`, `def`, `import`, `socket`, `$(...)`, `chmod +x` to your cheat-sheet.

Ethics — `sweep.sh` and `scanner.py` point at `127.0.0.1`, **your own machine, or the class lab only**. Scanning anything else is exactly the “no scope, no test” line from Day 1.

Real job — Why would a SOC team write essentially the same scanner a penetration tester writes, but aim it at their *own* estate instead?

Day 10 — Security thinking · in-class exercise, no separate homework

Security question: *Where does data from outside cross into somewhere powerful?*
— the question you’ll use for the rest of the course.

Do (in class) — the threat-model exercise: pick one app (result portal / food-delivery / UPI payment), draw it with trust boundaries, name 3 STRIDE threats, give each a control. Committed as `day10/threat-model.md` — no extra homework beyond finishing this in your repo if you didn’t in class.

Day 10 is also when the weekend assignment (below) is briefed — **Part E is not new work sprung on you today**. It’s due Monday but should already be well underway from Day 6.

Weekend assignment — Week 2

Submit **one PDF** before Monday (start of Day 11) for Parts A–D. Test **only your own VM**.

This is the integration exercise: harden a real machine using everything from Days 6–9, then threat-model and reflect on it — **plus** Part E, a separate, ongoing piece of evidence.

PART A — INTEGRATE (HARDEN YOUR MACHINE)

On your Linux VM:

1. **Threat-model it in half a page** — what's valuable on it, who'd realistically attack it, the entry points.
2. **Create a non-root user** and add it to a group; show `id`.
3. **Find and fix 3 weak settings** — e.g. a world-writable file, a `chmod 777`, a readable private key / `.env`. Show `ls -l` **before and after** each.
4. **Write a script** (Bash or Python — Day 9) that **reports** weak settings. Use `day10/assets/weak-settings-checklist.md` — at minimum, checks **1, 3, and 5** (world-writable files, unexpected SUID, readable secrets). It only needs to *report*.

PART B — R&D STRETCH (PICK ONE)

- **Linux:** research **one privilege-escalation technique we did *not* cover** (Linux capabilities, `LD_PRELOAD`, a specific GTF0Bin, PwnKit / CVE-2021-4034, dirtypipe, ...). In ~5 sentences: how it works and how a defender stops it.
- **Windows:** pick **3 LOLBins** (`certutil`, `regsvr32`, `mshta`, `bitsadmin`, `rundll32`, ...). What does each legitimately do, and why can't defenders simply block them?

PART C — HANDS-ON EVIDENCE

- your weak-settings script and its output;
- `ls -l` before/after for the 3 settings you fixed;
- `id` for the new non-root user.

PART D — REFLECTION

3–4 sentences: what clicked this week, what's still fuzzy.

PART E — OVERTHEWIRE BANDIT & COMMAND MASTERY

Submitted **separately from Parts A–D** — its own evidence set, not merged into the weekend PDF until your instructor asks for it. **Start this Day 6** — don't leave it for the weekend.

E1 — OverTheWire Bandit (minimum 10 levels). Work through the [OverTheWire Bandit wargame](#), starting at Level 0. **Minimum requirement: complete through Level 10.** Stretch goal (bonus): continue through **Level 15**.

- Keep a **Bandit Flag Log** as you go — one row per level: `Level | command(s)/technique used to find the password | flag obtained`.
- **Do not upload the flag log to Google Drive (or any shared location) until your instructor explicitly asks for it.** Keep it local — sharing flags early spoils the exercise for other students and defeats its use as a plagiarism check.

E2 — Linux & Windows command explainer. For **every command** in your `day06/linux-cheatsheet.md` and `day08/windows-cheatsheet.md`, run it for real on your own system (not copy-pasted from the slide) and write, per command:

1. The exact command you ran.
2. The output (screenshot or pasted text).
3. **In your own words** — what does this output tell you about the system, and what is this command actually used for? When would a sysadmin, SOC analyst, or attacker reach for it?

HOW THE WEEKEND ASSIGNMENT IS MARKED

Parts A–D (15 marks)

Area	What earns the marks	Marks
Part A1	half-page threat model of the VM: asset, actor, entry points	2
Part A2	non-root user created, <code>id</code> shown	1
Part A3	3 weak settings fixed, <code>ls -l</code> before/after for each	3
Part A4	script runs, reports checks 1/3/5 correctly, in your own code	4
Part B	real research beyond class, correct mechanism and defence	2
Part C	all three pieces of evidence present and legible	2
Part D	a genuine reflection	1

Part E (9 marks, scored separately)

Area	What earns the marks	Marks
E1 — Bandit levels	≥ 10 levels, flag log complete and correct per level	4
E1 — Bandit bonus	Levels 11–15 logged correctly	+2
E2 — command explainer	every cheat-sheet command run for real, output shown, explanation shows real understanding	3

Late or not one PDF for Parts A–D? Talk to your instructor before the deadline.

Submission checklist

In your repo (each folder holds the day's Do, Investigate, Secure and Evidence)

- ☐ `day06/` — cheat-sheet (15 commands), scavenger-hunt answers
- ☐ `day07/` — priv-esc artifact, `secret.txt` evidence, SUID answer
- ☐ `day08/` — Linux↔Windows command table, Run-key evidence, Windows cheat-sheet
- ☐ `day09/` — `scanner.py` + `sweep.sh`, `hosts.txt` extension, `ai_snippet.py` critique
- ☐ `day10/` — threat model
- ☐ Every day has a “Real job” answer and a “today I learned” entry

Handed in

- ☐ Weekend assignment, Parts A–D, in **one PDF**, before Monday
- ☐ Part E — Bandit Flag Log (kept **local**, not on Drive) and the command explainer write-up